Delivery Performance Optimization System Using Data Analytics

- **Group Members Name:**
  Rabia Sultan(20241-37153)
  Batool Zehra(20241-35829)
  Bisma Arshad(20231-34936)

- **Course Name:** Introduction to Database Management System
- **Course Code:** CSC - 217
- **Semester:** 4th - Sophomore
- **Instructor Name:** Muhammad Irtiza
- **Submission Date:** 14th

1. PROJECT TITLE

Delivery Performance Optimization System Using Data Analytics

2. INTRODUCTION

The Delivery Performance Optimization System is a database-driven solution developed to improve the operational efficiency of delivery and logistics companies. The system manages and analyzes data related to customers, orders, drivers, delivery zones, vehicles, and delivery tracking. It helps organizations monitor delivery operations and identify issues such as delays, traffic congestion, and inefficient driver management.

The system was developed using MySQL and focuses on data analytics and reporting. Through SQL queries, views, functions, and relational database design, the system provides meaningful business insights from operational data. The project also supports integration with a Streamlit-based dashboard where managers can visualize delivery performance, track risky deliveries, and analyze operational trends using charts and reports.

The main objective of the project is to convert raw delivery data into useful analytical information that helps management make better decisions and improve customer satisfaction.

3.PROBLEM STATEMENT

Delivery companies often face operational challenges that reduce efficiency and negatively affect customer experience. Many organizations still rely on manual tracking methods or poorly organized systems to manage deliveries, drivers, and customer orders. Because of this, management struggles to monitor delivery performance and identify the reasons behind delayed deliveries.

One major issue is traffic congestion in different delivery zones, which increases delivery time and creates delays. Another challenge is the lack of proper driver performance monitoring. Companies cannot easily determine which drivers are performing efficiently and which deliveries are at higher risk of delay.

In many cases, delivery tracking information is incomplete or scattered across multiple systems. This creates difficulties in analyzing operational performance and generating reports. Customers may also become dissatisfied due to delayed orders, lack of updates, or poor delivery service.

The organization needed a centralized database system that could:

- Store delivery-related data in an organized manner
- Track delivery activities
- Monitor driver and vehicle performance
- Analyze delayed and risky deliveries
- Generate operational reports and analytics

4.PROPOSED SOLUTION

To solve these problems, the Delivery Performance Optimization System was developed using MySQL database technologies and data analytics techniques. The proposed system centralizes all delivery-related information into a structured relational database.

The system stores and manages:

- Customer information
- Orders
- Delivery records
- Driver details
- Vehicle information
- Zone and congestion data
- Delivery tracking logs
- Customer feedback

Several SQL analytical features were implemented to improve decision-making and operational monitoring.

The system includes:

- Functions for calculating delays and driver workload
- Views for delayed deliveries and risky deliveries
- Queries for zone analysis and peak ordering time analysis
- Indexes for query optimization and faster performance

The project also supports dashboard visualization using Streamlit, where business users can view charts, delivery trends, and performance reports interactively.

The proposed system helps organizations:

- Improve delivery tracking
- Analyze delivery risks
- Monitor driver performance
- Improve customer satisfaction
- Reduce operational inefficiencies

5. STAKEHOLDERS

Stakeholders:

- **Business Owner**

The business owner wants better operational control, delivery tracking, and analytical reporting to improve company performance.

- **Admin**

The admin manages customer records, delivery records, drivers, and system operations.

- **Drivers**

Drivers are responsible for handling deliveries and updating delivery status.

- **Customers**

Customers place delivery orders and provide feedback after receiving deliveries.

- **Managers**

Managers monitor delivery performance, delays, and operational analytics.

6. BUSINESS REQUIREMENTS

The stakeholders requested the following features from the system:

- Store customer and order information
- Track deliveries in real time
- Maintain driver and vehicle records
- Analyze delayed deliveries
- Monitor delivery zones with high congestion
- Generate driver performance reports
- Identify risky deliveries automatically
- Maintain delivery logs and status history
- Provide customer feedback management
- Display analytical dashboards and reports

7. SOFTWARE AND TECHNOLOGIES USED

Software / Tool	Purpose
MySQL	Database Management System
MySQL Workbench	Database Design and ERD
SQL	Data Querying and Analytics
Streamlit	Dashboard and Data Visualization
Python	Backend Logic and Streamlit Integration
PYcharm	Python IDE

8. DATABASE DESIGN

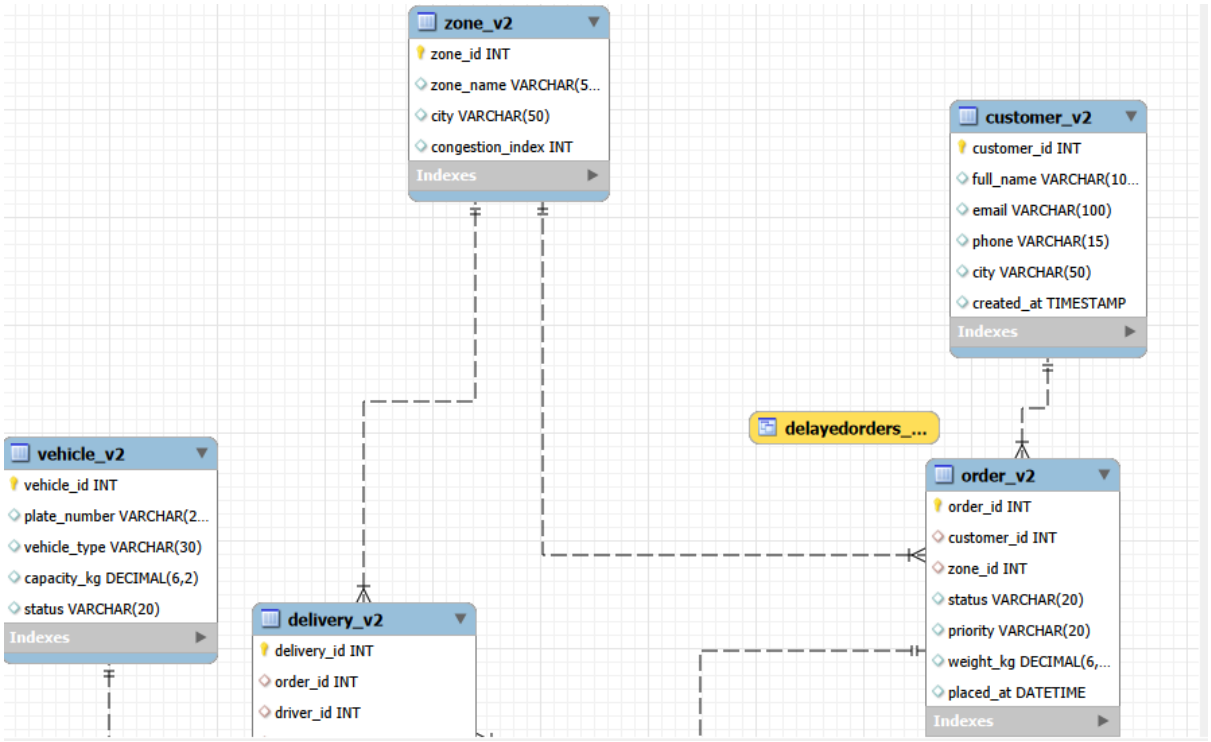
- **Entity Relationship Diagram (ERD)**

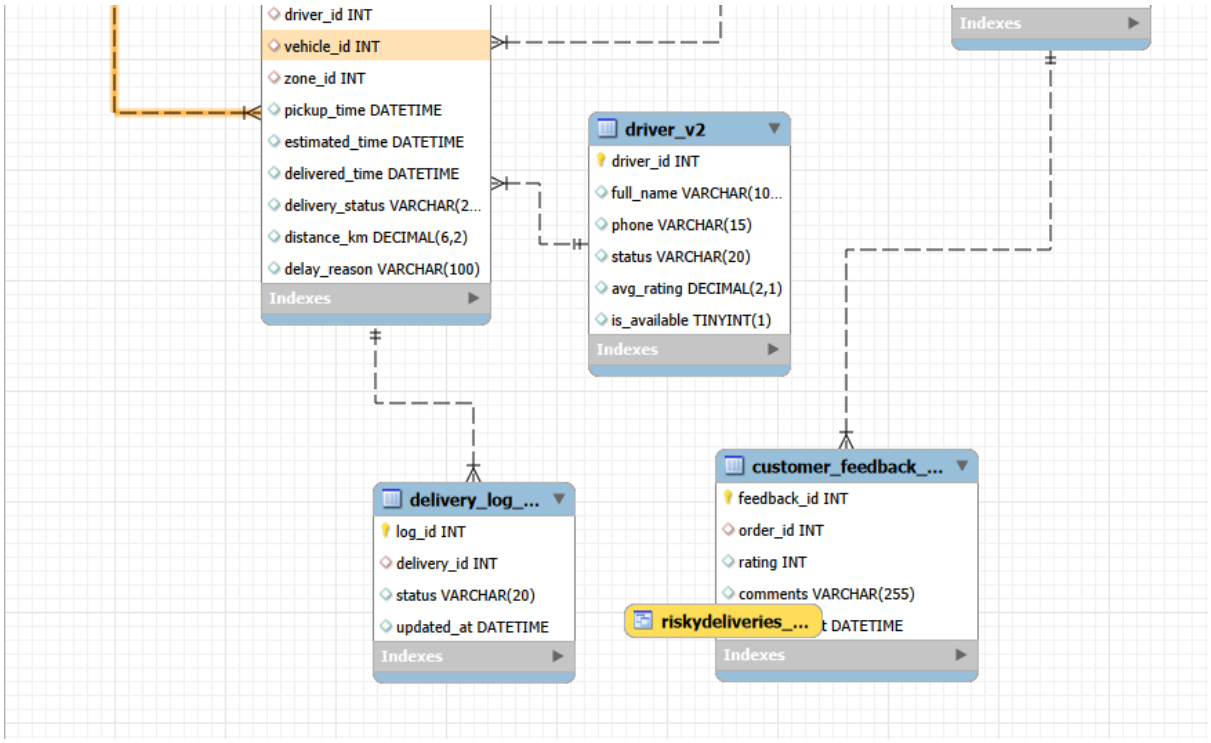
The database contains the following main entities:

- CUSTOMER
- ORDER
- DELIVERY
- DRIVER
- VEHICLE
- ZONE
- DELIVERY_LOG
- CUSTOMER_FEEDBACK

Relationship Summary

- One customer can place many orders
- One order belongs to one customer
- One order has one delivery
- One driver can handle many deliveries
- One vehicle can be used in many deliveries
- One zone can contain many orders and deliveries
- One delivery can have multiple delivery logs


zone_v2
zone_id INT
zone_name VARCHAR(5...
city VARCHAR(50)
congestion_index INT
Indexes
customer_v2
customer_id INT
full_name VARCHAR(10...
email VARCHAR(100)
phone VARCHAR(15)
city VARCHAR(50)
created_at TIMESTAMP
Indexes
delayedorders_...
vehicle_v2
vehicle_id INT
plate_number VARCHAR(2...
vehicle_type VARCHAR(30)
capacity_kg DECIMAL(6,2)
status VARCHAR(20)
Indexes
order_v2
order_id INT
customer_id INT
zone_id INT
status VARCHAR(20)
priority VARCHAR(20)
weight_kg DECIMAL(6,...
placed_at DATETIME
Indexes
delivery_v2
delivery_id INT
order_id INT
driver_id INT


driver_id INT
vehicle_id INT
zone_id INT
pickup_time DATETIME
estimated_time DATETIME
delivered_time DATETIME
delivery_status VARCHAR(2...
distance_km DECIMAL(6,2)
delay_reason VARCHAR(100)
Indexes
driver_v2
driver_id INT
full_name VARCHAR(10...
phone VARCHAR(15)
status VARCHAR(20)
avg_rating DECIMAL(2,1)
is_available TINYINT(1)
Indexes
Indexes
delivery_log_...
log_id INT
delivery_id INT
status VARCHAR(20)
updated_at DATETIME
Indexes
customer_feedback_...
feedback_id INT
order_id INT
rating INT
comments VARCHAR(255)
t DATETIME
Indexes
riskydeliveries_...

- # DATABASE TABLES

1.CUSTOMER_V2

Field Name	**Data Type**	**Constraint**
customer_id	INT	Primary Key
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE
phone	VARCHAR(15)	
city	VARCHAR(50)	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

2.ZONE_V2

Field Name	Data Type	Constraint
zone_id	INT	Primary Key
zone_name	VARCHAR(50)	
city	VARCHAR(50)	
congestion_index	INT	

3. DRIVER_V2

Field Name	Data Type	Constraint
driver_id	INT	Primary Key
full_name	VARCHAR(100)	
phone	VARCHAR(15)	
status	VARCHAR(20)	
avg_rating	DECIMAL(2,1)	
is_available	BOOLEAN	

4. VEHICLE_V2

Field Name	Data Type	Constraint
vehicle_id	INT	Primary Key
plate_number	VARCHAR(20)	
vehicle_type	VARCHAR(30)	
capacity_kg	DECIMAL(6,2)	
status	VARCHAR(20)	

5. ORDER_V2

Field Name	Data Type	Constraint
order_id	INT	Primary Key
customer_id	INT	Foreign Key references CUSTOMER_V2(customer_id)
zone_id	INT	Foreign Key references ZONE_V2(zone_id)
status	VARCHAR(20)	
priority	VARCHAR(20)	
weight_kg	DECIMAL(6,2)	
placed_at	DATETIME	

6. DELIVERY_V2

Field Name	Data Type	Constraint
delivery_id	INT	Primary Key
order_id	INT	UNIQUE, Foreign Key references ORDER_V2(order_id)
driver_id	INT	Foreign Key references DRIVER_V2(driver_id)
vehicle_id	INT	Foreign Key references VEHICLE_V2(vehicle_id)
zone_id	INT	Foreign Key references ZONE_V2(zone_id)
pickup_time	DATETIME	
estimated_time	DATETIME	
delivered_time	DATETIME	
delivery_status	VARCHAR(20)	
distance_km	DECIMAL(6,2)	
delay_reason	VARCHAR(100)	

7. DELIVERY_LOG_V2

Field Name	Data Type	Constraint
log_id	INT	Primary Key
delivery_id	INT	Foreign Key references DELIVERY_V2(delivery_id)
status	VARCHAR(20)	
updated_at	DATETIME	

8. CUSTOMER_FEEDBACK_V2

Field Name	Data Type	Constraint
feedback_id	INT	Primary Key
order_id	INT	Foreign Key references ORDER_V2(order_id)
rating	INT	CHECK (rating BETWEEN 1 AND 5)
comments	VARCHAR(255)	
submitted_at	DATETIME	

9. SQL QUERIES AND BUSINESS PROBLEM SOLUTION

- **Query 1: Delivery Delay Analysis**

Business Problem

Management wanted to monitor delayed deliveries.

SQL Query

```
SELECT
   delivery_id,
   get_delay_minutes_v2(delivery_id) AS delay_minutes
FROM DELIVERY_V2;
```

Result

The query calculated delivery delay time in minutes.

Business Impact

Helped management identify delayed deliveries and monitor operational efficiency.

- **Query 2: Driver Performance Analysis**

Business Problem

The company wanted to analyze driver workload and performance.

SQL Query

```
SELECT
   driver_id,
   get_driver_total_v2(driver_id) AS total_deliveries
FROM DRIVER_V2;
```

Result

Displayed the total number of deliveries completed by each driver.

Business Impact

Helped management evaluate driver productivity and workload distribution.

- **Query 3: Zone Delay Analysis**

Business Problem

The company wanted to identify zones with high delivery delays.

SQL Query

```
SELECT
   z.zone_name,
   COUNT(*) AS delayed_orders
FROM DELIVERY_V2 d
INNER JOIN ZONE_V2 z
ON d.zone_id = z.zone_id
WHERE d.delivered_time > d.estimated_time
GROUP BY z.zone_name;
```

Result

Displayed the number of delayed deliveries in each zone.

Business Impact

Helped management identify traffic-prone zones and improve route planning.

- **Query 4: Peak Ordering Time Analysis**

Business Problem

The company wanted to identify busy order hours.

SQL Query

```
SELECT
   HOUR(placed_at) AS hour,
   COUNT(*) AS total_orders
FROM ORDER_V2
GROUP BY HOUR(placed_at);
```

Result

Displayed the busiest ordering hours of the day.

Business Impact

Helped management schedule drivers and allocate resources more effectively.

10.VIEWS USED IN THE SYSTEM

- **DelayedOrders_V2**

Displays only delayed deliveries.

- **DriverPerformance_V2**

Calculates average delivery time and total deliveries per driver.

- **RiskyDeliveries_V2**

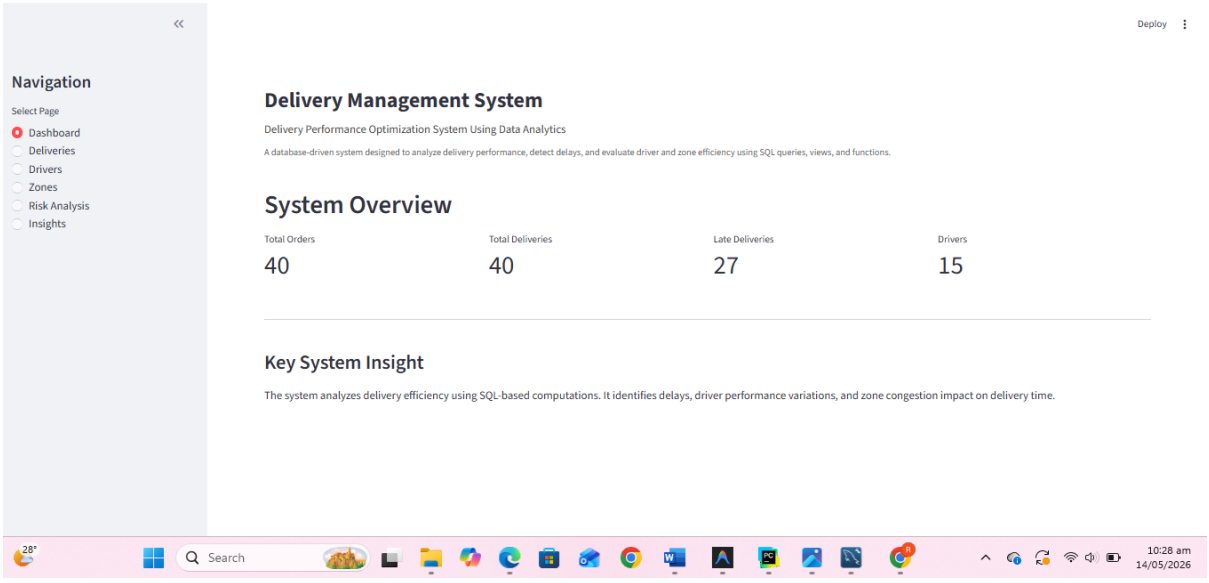
Identifies high-risk deliveries using:

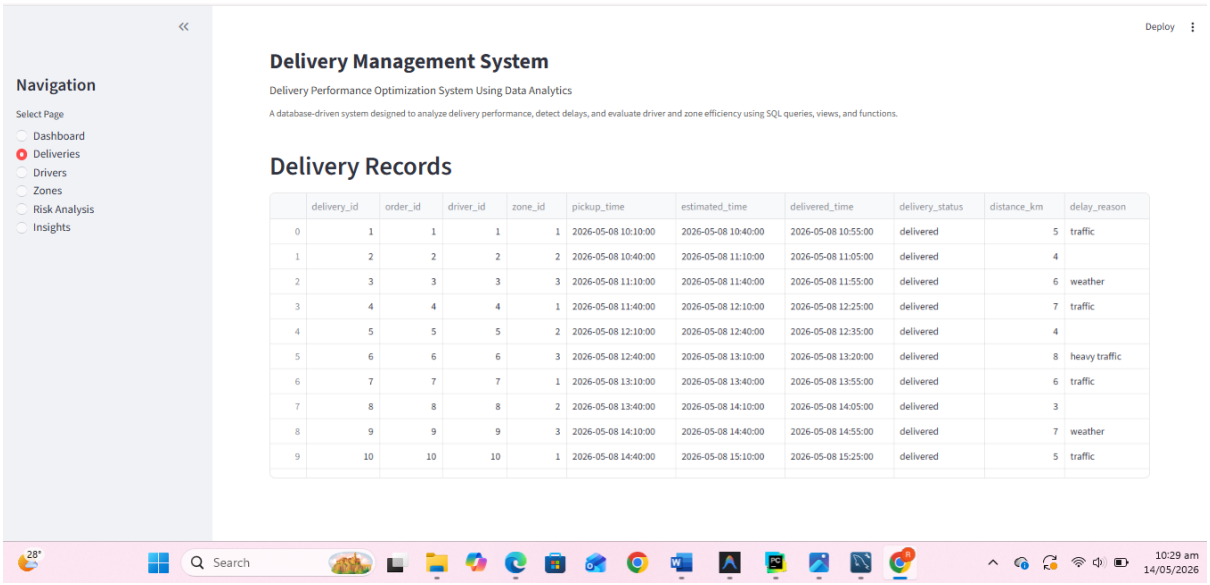
- congestion index
- average delivery time
- delivery distance

This view helps management predict risky operations and improve planning.

11. WEBSITE/SYSTEM INTERFACE

- **Website screenshot**





	delivery_id	order_id	driver_id	zone_id	pickup_time	estimated_time	delivered_time	delivery_status	distance_km	delay_reason
0	1	1	1	1	2026-05-08 10:10:00	2026-05-08 10:40:00	2026-05-08 10:55:00	delivered	5	traffic
1	2	2	2	2	2026-05-08 10:40:00	2026-05-08 11:10:00	2026-05-08 11:05:00	delivered	4	
2	3	3	3	3	2026-05-08 11:10:00	2026-05-08 11:40:00	2026-05-08 11:55:00	delivered	6	weather
3	4	4	4	1	2026-05-08 11:40:00	2026-05-08 12:10:00	2026-05-08 12:25:00	delivered	7	traffic
4	5	5	5	2	2026-05-08 12:10:00	2026-05-08 12:40:00	2026-05-08 12:35:00	delivered	4	
5	6	6	6	3	2026-05-08 12:40:00	2026-05-08 13:10:00	2026-05-08 13:20:00	delivered	8	heavy traffic
6	7	7	7	1	2026-05-08 13:10:00	2026-05-08 13:40:00	2026-05-08 13:55:00	delivered	6	traffic
7	8	8	8	2	2026-05-08 13:40:00	2026-05-08 14:10:00	2026-05-08 14:05:00	delivered	3	
8	9	9	9	3	2026-05-08 14:10:00	2026-05-08 14:40:00	2026-05-08 14:55:00	delivered	7	weather
9	10	10	10	1	2026-05-08 14:40:00	2026-05-08 15:10:00	2026-05-08 15:25:00	delivered	5	traffic

Deploy

Navigation

Select Page

- Dashboard
- Deliveries
- Drivers
- Zones
- Risk Analysis
- Insights

Delivery Management System

Delivery Performance Optimization System Using Data Analytics

A database-driven system designed to analyze delivery performance, detect delays, and evaluate driver and zone efficiency using SQL queries, views, and functions.

Driver Performance Analysis

Driver Details

	driver_id	full_name	avg_rating	is_available
0	1	Usman	4.5	1
1	2	Bilal	4.2	1
2	3	Hassan	4.8	1
3	4	Waqas	4.1	1
4	5	Asad	4.3	1
5	6	Rizwan	4.7	1
6	7	Haris	4	1
7	8	Junaid	4.6	1
8	9	Naeem	4.4	1

Deploy

9	10	Salman	4.5	1

Performance Summary (SQL-based)

	driver_id	total_deliveries	avg_delivery_time
0	2	3	25
1	5	3	25
2	8	3	25
3	11	2	25
4	14	2	25
5	6	3	43.3333
6	1	3	45
7	3	3	45
8	4	3	45
9	7	3	45

Deploy

Navigation

Select Page

- Dashboard
- Deliveries
- Drivers
- Zones
- Risk Analysis
- Insights

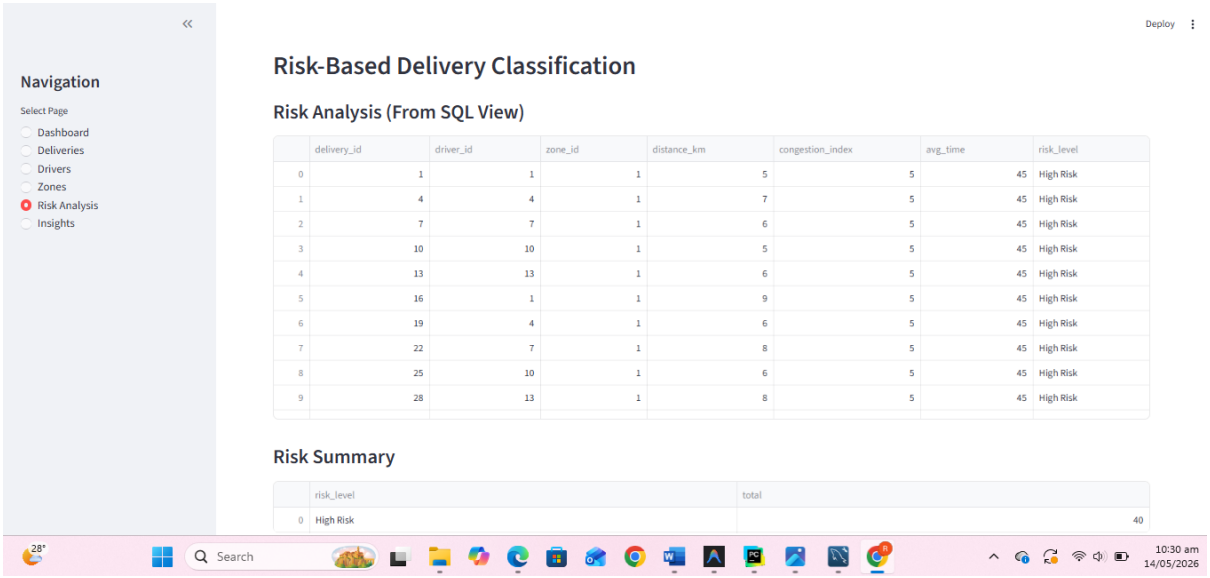
Delivery Management System

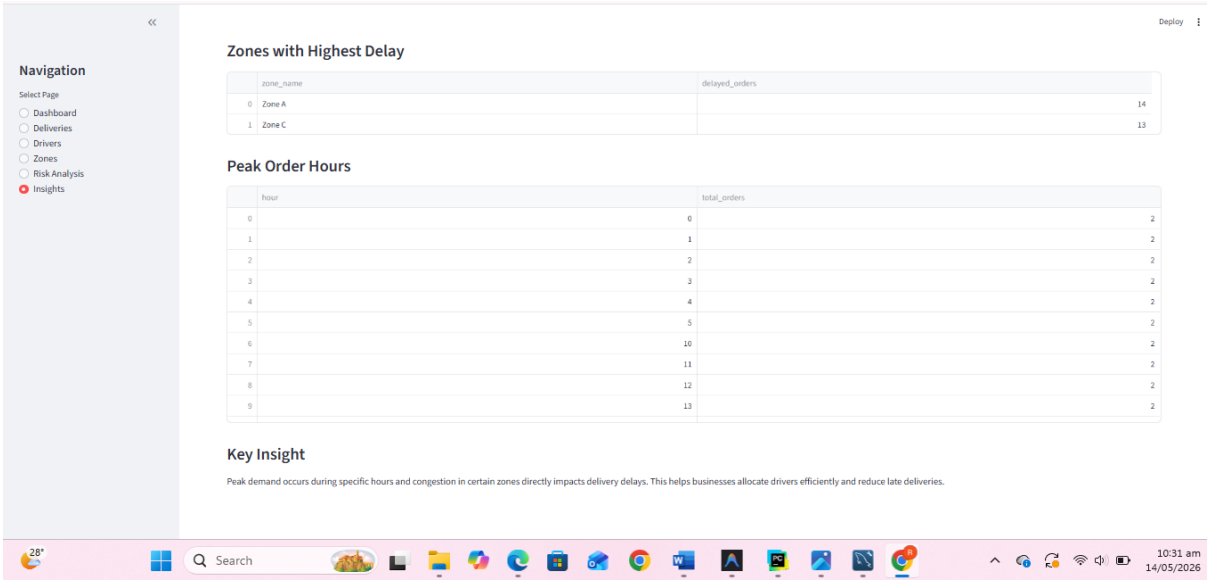
Delivery Performance Optimization System Using Data Analytics

A database-driven system designed to analyze delivery performance, detect delays, and evaluate driver and zone efficiency using SQL queries, views, and functions.

Zone Performance Analysis

	zone_name	city	congestion_index	total_deliveries	avg_delay
0	Zone A	Lahore	5	14	15
1	Zone B	Karachi	7	13	-5
2	Zone C	Islamabad	8	13	14.6154





- **Website Link:**
  (local streamlit host, app.py code)

```
import mysql.connector

import streamlit as st
import mysql.connector
import pandas as pd
import os

# ---------------- PAGE CONFIG ---------------- #

st.set_page_config(
```

```
    page_title="Delivery Management System",
    layout="wide"
)

# ---------------- DATABASE CONNECTION ---------------- #

@st.cache_resource
def get_connection():
    return mysql.connector.connect(
        host=os.getenv("DB_HOST", "localhost"),
        user=os.getenv("DB_USER", "root"),
        password=os.getenv("DB_PASSWORD", ""),
        database=os.getenv("DB_NAME", "delivery_db")
    )

def run_query(query):
    """Run a SQL query and return a DataFrame. Reconnects if connection was
lost."""
    conn = get_connection()
    try:
        conn.ping(reconnect=True)  # re-establish connection if dropped
        return pd.read_sql(query, conn)
    except mysql.connector.Error as e:
        st.error(f"Database error: {e}")
        return pd.DataFrame()

# ---------------- HEADER ---------------- #

st.markdown("""
    <style>
        .main-title {
            font-size: 30px;
            font-weight: 700;
            margin-bottom: 5px;
        }
        .subtitle {
            font-size: 16px;
            color: #444;
            margin-bottom: 10px;
        }
        .purpose {
            font-size: 13px;
            color: #666;
            margin-bottom: 25px;
            line-height: 1.5;
        }
        .box {
            padding: 15px;
            border: 1px solid #ddd;
            border-radius: 8px;
            background-color: #fafafa;
        }
    </style>
""", unsafe_allow_html=True)

st.markdown('<div class="main-title">Delivery Management System</div>',
unsafe_allow_html=True)

st.markdown(
    '<div class="subtitle">Delivery Performance Optimization System Using
Data Analytics</div>',
```

```
    unsafe_allow_html=True
)

st.markdown(
    '<div class="purpose">A database-driven system designed to analyze
delivery performance, detect delays, and evaluate driver and zone
efficiency using SQL queries, views, and functions.</div>',
    unsafe_allow_html=True
)

# ---------------- SIDEBAR ---------------- #

st.sidebar.title("Navigation")

page = st.sidebar.radio(
    "Select Page",
    ["Dashboard", "Deliveries", "Drivers", "Zones", "Risk Analysis",
"Insights"]
)

# ---------------- DASHBOARD ---------------- #

if page == "Dashboard":

    st.header("System Overview")

    col1, col2, col3, col4 = st.columns(4)

    total_orders = run_query("SELECT COUNT(*) AS c FROM ORDER_V2")
    total_deliveries = run_query("SELECT COUNT(*) AS c FROM DELIVERY_V2")
    late_deliveries = run_query("SELECT COUNT(*) AS c FROM DELIVERY_V2
WHERE delivered_time > estimated_time")
    total_drivers = run_query("SELECT COUNT(*) AS c FROM DRIVER_V2")

    col1.metric("Total Orders", int(total_orders["c"][0]) if not
total_orders.empty else 0)
    col2.metric("Total Deliveries", int(total_deliveries["c"][0]) if not
total_deliveries.empty else 0)
    col3.metric("Late Deliveries", int(late_deliveries["c"][0]) if not
late_deliveries.empty else 0)
    col4.metric("Drivers", int(total_drivers["c"][0]) if not
total_drivers.empty else 0)

    st.divider()

    st.subheader("Key System Insight")

    st.write("""
    The system analyzes delivery efficiency using SQL-based computations.
    It identifies delays, driver performance variations, and zone
congestion impact on delivery time.
    """)

# ---------------- DELIVERIES ---------------- #

elif page == "Deliveries":

    st.header("Delivery Records")

    df = run_query("""
        SELECT delivery_id, order_id, driver_id, zone_id,
```

```
                pickup_time, estimated_time, delivered_time,
                delivery_status, distance_km, delay_reason
            FROM DELIVERY_V2
        """)

        st.dataframe(df, use_container_width=True)

    # ---------------- DRIVERS ---------------- #

    elif page == "Drivers":

        st.header("Driver Performance Analysis")

        st.subheader("Driver Details")

        drivers = run_query("""
            SELECT driver_id, full_name, avg_rating, is_available
            FROM DRIVER_V2
        """)

        st.dataframe(drivers, use_container_width=True)

        st.subheader("Performance Summary (SQL-based)")

        performance = run_query("""
            SELECT driver_id,
                COUNT(*) AS total_deliveries,
                AVG(TIMESTAMPDIFF(MINUTE, pickup_time, delivered_time)) AS
    avg_delivery_time
            FROM DELIVERY_V2
            GROUP BY driver_id
            ORDER BY avg_delivery_time ASC
        """)

        st.dataframe(performance, use_container_width=True)

    # ---------------- ZONES ---------------- #

    elif page == "Zones":

        st.header("Zone Performance Analysis")

        zones = run_query("""
            SELECT z.zone_name,
                z.city,
                z.congestion_index,
                COUNT(d.delivery_id) AS total_deliveries,
                AVG(TIMESTAMPDIFF(MINUTE, d.estimated_time,
    d.delivered_time)) AS avg_delay
            FROM ZONE_V2 z
            LEFT JOIN DELIVERY_V2 d ON z.zone_id = d.zone_id
            GROUP BY z.zone_id
        """)

        st.dataframe(zones, use_container_width=True)

    # ---------------- RISK ANALYSIS ---------------- #

    elif page == "Risk Analysis":

        st.header("Risk-Based Delivery Classification")
```

```
    st.subheader("Risk Analysis (From SQL View)")

    risk = run_query("SELECT * FROM RiskyDeliveries_V2")

    st.dataframe(risk, use_container_width=True)

    st.subheader("Risk Summary")

    risk_summary = run_query("""
        SELECT risk_level, COUNT(*) AS total
        FROM RiskyDeliveries_V2
        GROUP BY risk_level
    """)

    st.dataframe(risk_summary, use_container_width=True)

# ---------------- INSIGHTS ---------------- #

elif page == "Insights":

    st.header("Business Insights (SQL Analysis)")

    st.subheader("Zones with Highest Delay")

    delays = run_query("""
        SELECT z.zone_name,
               COUNT(*) AS delayed_orders
        FROM DELIVERY_V2 d
        JOIN ZONE_V2 z ON d.zone_id = z.zone_id
        WHERE d.delivered_time > d.estimated_time
        GROUP BY z.zone_name
        ORDER BY delayed_orders DESC
    """)

    st.dataframe(delays, use_container_width=True)

    st.subheader("Peak Order Hours")

    peak = run_query("""
        SELECT HOUR(placed_at) AS hour,
               COUNT(*) AS total_orders
        FROM ORDER_V2
        GROUP BY HOUR(placed_at)
        ORDER BY hour
    """)

    st.dataframe(peak, use_container_width=True)

    st.subheader("Key Insight")

    st.write("""
    Peak demand occurs during specific hours and congestion in certain
zones directly impacts delivery delays.
    This helps businesses allocate drivers efficiently and reduce late
deliveries.
    """)
```

12. CHALLENGES FACED DURING DEVELOPMENT

Several challenges were faced during the development of the project.

- **Database Relationship Design**

Designing proper relationships between entities such as orders, deliveries, drivers, and customers required careful planning to maintain normalization and avoid redundancy.

- **Query Optimization**

Some analytical queries initially executed slowly when joins were used on large tables. Indexes were later created on frequently searched columns to improve performance.

- **ERD Complexity**

While reverse engineering the database in MySQL Workbench, many indirect relationships appeared connected, making the ERD difficult to understand initially.

- **Delay Calculation Logic**

Calculating delivery delays accurately using timestamps required proper use of MySQL date and time functions.

- **Dashboard Integration**

Integrating SQL analytics with Streamlit charts required proper query formatting and data handling.

All these issues were gradually resolved through testing, debugging, normalization, and query optimization.

CONCLUSION

The Delivery Performance Optimization System successfully provides a centralized and analytical solution for managing delivery operations. The project demonstrates how database systems and SQL analytics can be used to improve operational efficiency, monitor delivery performance, and support business decision-making.

Using MySQL functions, views, joins, indexes, and analytical queries, the system transforms raw delivery data into meaningful business insights. The project also demonstrates the practical implementation of database concepts such as normalization, foreign keys, constraints, ERD modeling, and performance optimization.

Overall, the system provides a strong foundation for modern logistics and delivery management solutions.